

Relational Database Systems: ACID Transactions & Concurrency Control

Course: CS304 Database Architecture | License: CC-BY 4.0 Open Educational Resource

1. The ACID Paradigm of Database Transactions

A transaction is an execution unit of program code that accesses and potentially updates various database items. To preserve integrity, the DBMS guarantees the four ACID properties: Atomicity guarantees all-or-nothing execution; if any operation aborts, prior changes are undone via transaction rollback. Consistency ensures that a transaction transforms the database from one valid state satisfying all schema constraints and invariants to another valid state. Isolation ensures that concurrent transactions execute without interfering with one another, as though executed in isolation. Durability ensures that once a transaction successfully commits, its updates persist permanently on non-volatile storage even across power failures.

2. Serializability and Concurrency Anomalies

Concurrent execution schedules are deemed correct if they are serializable—producing the exact database state resulting from some purely serial execution of transactions. Conflict serializability evaluates whether non-conflicting adjacent operations can be swapped into a serial schedule. Two operations conflict if they belong to different transactions, access the same item, and at least one is a write. Standard ANSI SQL isolation levels protect against specific read anomalies: Dirty Read occurs when transaction T2 reads uncommitted modifications made by T1 that are subsequently rolled back. Non-Repeatable Read occurs when T1 re-reads a row and finds altered data because T2 committed an update in between. Phantom Read occurs when T1 re-executes a range query and discovers new rows inserted by committed transaction T2.

3. Two-Phase Locking (2PL) Protocols

Two-Phase Locking (2PL) is a concurrency control protocol that guarantees conflict serializability. The protocol enforces two rules: 1) Growing Phase: A transaction may acquire locks (shared or exclusive) but cannot release any lock. 2) Shrinking Phase: A transaction may release locks but cannot acquire any new locks. Shared locks (S-locks) permit concurrent reads; exclusive locks (X-locks) grant sole read-write privileges.

Standard 2PL does not prevent cascading aborts. Strict Two-Phase Locking (Strict 2PL) solves this by requiring all exclusive locks to be held until the transaction finishes (commit or abort). Rigorous 2PL goes further by holding both shared and exclusive locks until the transaction terminates, ensuring strict schedules and simplifying rollback.

4. Deadlocks in Database Management Systems

Because transactions acquire locks incrementally, deadlocks can occur when two transactions wait for locks held by each other. DBMS engines employ two strategies: Deadlock Prevention and Deadlock Detection. Deadlock prevention protocols use transaction timestamps to avoid wait cycles before they occur: Wait-Die Scheme (Non-preemptive): If an older transaction requests a lock held by a younger transaction, the older is allowed to wait. If a younger transaction requests a lock held by an older transaction, the younger dies (is rolled back and restarted). Wound-Wait Scheme (Preemptive): If an older transaction requests a lock held by a younger transaction, the older wounds the younger, forcing it to abort and yield the lock. If a younger requests from an older, the younger is allowed to wait.

5. Deadlock Detection via Wait-For Graphs

Deadlock detection periodically constructs a directed Wait-For Graph (WFG) where vertices represent active transactions and directed edges ($T_i \rightarrow T_j$) indicate that T_i is waiting for T_j to release a lock. A cycle in the WFG signals a deadlock. Upon detecting a cycle, the DBMS selects a victim transaction based on rollback cost, CPU time consumed, or lock count, and aborts it to break the dependency loop.

6. Write-Ahead Logging (WAL) and ARIES Recovery

The Write-Ahead Logging (WAL) protocol dictates that log records describing a modification must be flushed to non-volatile disk before the corresponding dirty data page is written to database storage. Furthermore, a transaction is not considered safely committed until its COMMIT log record is forced to stable log storage. The ARIES (Algorithm for Recovery and Isolation Exploiting Semantics) algorithm executes three passes during crash recovery: 1. Analysis Pass: Identifies all dirty pages in buffer pool and active uncommitted transactions at crash time. 2. Redo Pass: Repeats history starting from the smallest dirty page LSN (Log Sequence Number) to restore the exact state prior to crash, redoing both committed and uncommitted operations. 3. Undo Pass: Reverses the actions of all active (loser) transactions in reverse chronological order, writing Compensation Log Records (CLRs) to ensure crash-safe rollback.